Unit 1 Review

Everything here is from Unit 1. Find the skill you need, then write your own version for your data.

Lists

Positions and length. The first item is position 0. A negative position counts back from the end.

```
pets = ["cat", "dog", "fish", "frog"]
print(pets[0])     # cat
print(pets[-1])    # frog
print(len(pets))   # 4
```

Slices. A slice stops just before its end number. A missing start or end means the front or the back.

```
print(pets[1:3])   # ['dog', 'fish']
print(pets[:2])    # ['cat', 'dog']
print(pets[::2])   # ['cat', 'fish']
pets[1:3] = ["bird"]   # replaces dog and fish
```

Growing and shrinking. `.remove()` takes a value. `.pop()` takes a position.

```
pets.append("hamster")
pets.remove("dog")
first = pets.pop(0)
```

Is it in the list? `.strip()` trims spaces off the ends, and `.lower()` makes it lower case.

```
answer = input("Favorite pet? ")
if answer.strip().lower() in pets:
    print("We have one!")
else:
    print("Not here")
```

Loops

A loop with an if inside. The indented lines run once for every item.

```
for pet in pets:
    if len(pet) == 4:
        print(pet)
```

Counting in a loop. Start at 0, add 1 when the test passes, print after the loop.

```
count = 0
for pet in pets:
    if "o" in pet:
        count = count + 1
print(count, "pets have an o")
```

Strings work like lists. Reach a letter by position, or loop over the letters.

```
word = "otter"
print(word[0], word[-1])    # o r
for letter in word:
    if letter == "t":
        print("found a t")
```

A True/False flag. Start `True`. Any item that fails switches it to `False`.

```
safe = True
for food in ["rice", "peanuts", "apple"]:
    if food == "peanuts":
        safe = False
print("Safe to eat:", safe)
```

Two lists that line up. `range(len())` gives the positions. `.index()` finds where an item is.

```
names = ["Ana", "Ben", "Cleo"]
pets = ["cat", "dog", "fish"]
for i in range(len(names)):
    print(names[i], "has a", pets[i])
print(pets[names.index("Ben")])   # dog
```

Loop until you stop it. `break` ends a `while True:` loop. Stuck? Click the terminal, press Ctrl+C.

```
toys = []
while True:
    toy = input("Toy? (done to stop) ")
    if toy == "done":
        break
    toys.append(toy)
```

Files

Read a text file. `.split()` with nothing in the brackets splits on spaces and new lines.

```
with open("notes.txt") as f:
    text = f.read()
words = text.split()
print(len(words), "words")
```

Read a CSV. Split into lines, keep the header apart, split each row on commas.

```
with open("pets.csv") as f:
    lines = f.read().strip().split("\n")
header = lines[0].split(",")
rows = lines[1:]
for row in rows:
    fields = row.split(",")
```

Text into numbers. A file and `input()` give text. `int()` makes a whole number, `float()` a decimal.

```
parts = "Leo,13,5.5".split(",")
age = int(parts[1])
height = float(parts[2])
n = int(input("How many? "))
```

Bugs we hunted: a missing quote, a missing `:`, a line that should be indented, `=` where `==` belongs, `range(6)` on a list of 5, `.remove()` where `.append()` belongs. Read the error's line number first.

Answering questions with data

Biggest so far. Keep the best so far, and replace it when a bigger one comes along.

```
biggest = 0
for t in [61, 75, 58, 80]:
    if t > biggest:
        biggest = t
print("Hottest:", biggest)
```

Build a list as you go. Start empty, `.append()` the ones that pass. `and` needs both tests.

```
warm = []
for t in [61, 75, 88, 70]:
    if t > 65 and t < 85:
        warm.append(t)
print(len(warm), "warm days:", warm)
```

An average. Add up a total and a count in the loop, then divide after it.

```
total = 0
count = 0
for t in [61, 75, 88, 70]:
    total = total + t
    count = count + 1
print("Average temp:", total / count)
```

The median (Spicy). Sort the list, then take the middle one. `//` divides and drops the remainder.

```
temps = sorted([70, 58, 81, 64, 77])
middle = len(temps) // 2
print("Median temp:", temps[middle])   # 70
```

Dictionaries

Count how many of each. `.get(key, 0)` gives 0 when the key is not there yet.

```
counts = {}
for pet in ["cat", "dog", "cat"]:
    counts[pet] = counts.get(pet, 0) + 1
for pet in counts:
    print(pet, counts[pet])
```

A lookup table. Build it once, then look things up while you loop.

```
city = {"BOS": "Boston", "LAX": "LA"}
for code in ["BOS", "JFK"]:
    print(code, city.get(code, "unknown"))
```

Functions

Parameters. The values in the call fill the names in the `def`, in order.

```
def invite(name, day):
    print(name, "come over on", day)
invite("Leo", "Friday")
```

Return. `return` hands a value back. A function can call another one and use its answer.

```
def tax(price):
    return price * 0.06
def total(price):
    return price + tax(price)
print(total(20))   # 21.2
```

Random and str(). `str()` turns a number into text so `+` can join them.

```
import random
color = random.choice(["red", "blue"])
n = random.randint(1, 6)
print(color + str(n))
```

Recursion. A function calls itself on a smaller copy of the problem.

```
def stars(n):
    if n == 0:
        return ""
    return "*" + stars(n - 1)
```

Charts

Start with these three lines. There is no chart window, so charts save to a file.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

Bar chart. Compares groups. It takes a list of labels and a list of heights that line up.

```
plt.bar(["cats", "dogs"], [12, 30])
plt.title("Favorite pets")
plt.xlabel("Pet")
plt.ylabel("Votes")
plt.savefig("pets.png")
```

Scatter plot. Shows two number columns together. `plt.clf()` clears the last chart first. Open each `.png` from the file tree to check it.

```
plt.clf()
plt.scatter([1, 2, 3, 4], [2, 4, 5, 8])
plt.xlabel("Hours of sun")
plt.ylabel("Plant height")
plt.savefig("plants.png")
```